

MODEL QUESTION PAPER
RELATIONAL DATABASE MANAGEMENT SYSTEM - SET I

(Maximum marks: 75)

(Time: 3 hours)

PART A

I. Answer all questions in one word or one sentence. Each question carries one mark.

(9 x 1 = 9 marks)

1	Define database instance.	M 1.01	R
2	State Schema.	M 1.01	R
3	Give the definition for Domain	M 2.01	R
4	Define Tuple	M 2.01	R
5	Define SQL	M 3.01	R
6	State the syntax of CREATE TABLE command	M 3.02	A
7	Define Trigger	M 4.01	R
8	Define Cursor	M 4.01	R
9	Define Primary key	M 1.03	R

PART B

II Answer any eight questions from the following. Each question carries 3 marks

(8 x 3 = 24marks)

1	Explain any three advantages of DBMS	M 1.01	U
2	Explain Two tier client/server architecture with diagram	M 1.02	U
3	Distinguish between Physical record and Logical Record	M 1.01	U
4	Explain domain constraints and key constraints	M 2.01	U
5	Elaborate on :(a) Dependency preservation (b) Decomposition	M 2.04	A
6	Discuss any three design guidelines for relational schema	M 2.03	A
7	Explain any three Data types in SQL	M 3.01	U
8	Elaborate on :(a) CREATE INDEX (b) DROP INDEX	M 3.02	A
9	Explain Concurrent Executions	M 4.03	U
10	Elaborate on the following transaction states (i) Committed (ii) Aborted	M 4.03	U

PART C

Answer all questions. Each question carries seven marks

(6x7=42 marks)

III	Elaborate on(i) Database Users (ii) Query Processor	M 1.02	U								
IV	OR Draw ER diagram for the relation published by associating the two schemes as given below: (i) Book (ISBN, TITLE, CATEGORY, PRICE, YEAR, AUTHOR, PAGE_COUNT, PID) (ii) Publisher (PID, PNAME, ADDRESS, STATE, PHONE, MAIL ID)	M 1.03	A								
V	Find union, intersection and difference for the given tables	M2.02	U								
<table border="1" style="width: 100%; border-collapse: collapse;"> <thead> <tr> <th colspan="2" style="text-align: center;">Table I</th> <th colspan="2" style="text-align: center;">Table II</th> </tr> </thead> <tbody> <tr> <td style="text-align: center;">Pub Id</td> <td style="text-align: center;">Pub Name</td> <td style="text-align: center;">Pub Id</td> <td style="text-align: center;">Pub Name</td> </tr> </tbody> </table>				Table I		Table II		Pub Id	Pub Name	Pub Id	Pub Name
Table I		Table II									
Pub Id	Pub Name	Pub Id	Pub Name								

VI					M2.02	U
	P01	Pearson Education	P01	Pearson Education		
	P02	Tata McGraw Hill	P02	Tata McGraw Hill		
	P03	Prentice Hall India	P05	Penram International		
	P04	Elsevier	P06	Bright Publication		
OR Explain the following relational algebra operations: (i) Select (ii) Rename (iii) Cartesian product						
VII	Describe the concept of nested queries in SQL with example.				M3.02	A
VIII	OR Write SQL queries for the following: (Table stores the details of a student) Create a table 'STUDENT' with attributes: Regno, name, branch, semester, mark and grade. (ii) Retrieve the details of students studying in semester 54. (iii) Insert regno, name, branch and semester of a student. (iv) Update grade of a student with regno '1111'to 'B'. (v) Display maximum and minimum mark of student from each branch, (vi) Retrieve regno, name, branch and semester of a student with highest mark.				M3.02	A
IX	Explain stored procedures and functions				M4.01	U
X	OR Explain the application of Embedded SQL statements				M4.02	U
XI	Explain Serializability				M4.03	U
XII	OR Explain the ACID properties of transaction.				M4.03	U
XIII	Specify symbols and the meaning of ER Model Notations.				M1.03	U
XIV	OR Describe on relationship types in ER model				M1.03	U

Question Wise Analysis

Q. No	Module Outcome	Cognitive Level	Marks	Time
I.1	M 1.01	R	1	1 min
I.2	M 1.01	R	1	1 min
I.3	M 2.01	R	1	1 min
I.4	M 2.01	R	1	1 min
I.5	M 3.01	R	1	1 min
I.6	M 3.02	A	1	1 min
I.7	M 4.01	R	1	1 min
I.8	M 4.01	R	1	1 min
I.9	M 1.03	R	1	1 min
II.1	M 1.01	U	3	6 min
II.2	M 1.02	U	3	6 min
II.3	M 1.01	U	3	6 min
II.4	M 2.01	U	3	6 min
II.5	M 2.04	A	3	6 min
II.6	M 2.03	A	3	6 min
II.7	M 3.01	U	3	6 min
II.8	M 3.02	A	3	6 min
II.9	M 4.03	U	3	6 min
II.10	M 4.03	U	3	6 min
III	M 1.02	U	7	20 min
IV	M 1.03	A	7	20 min
V	M 2.02	U	7	20 min
VI	M 2.02	U	7	20 min
VII	M 3.02	A	7	20 min
VIII	M 3.02	A	7	20 min
IX	M 4.01	U	7	20 min
X	M 4.02	U	7	20 min

XI	M 4.03	U	7	20 min
XII	M 4.03	U	7	20 min
XIII	M 1.03	U	7	20 min
XIV	M 1.03	U	7	20 min

Course : Relational Database Management System

Blue Print

Mark Distribution

Module	hr / Module	Marks/Module ($h_i / \sum H_i$) * 123 ($\pm 5\%$)	Type of Questions							
			Part A		Part B		Part C		Total	
			No of Questions	Marks	No of Questions	Marks	No of Questions	Marks	No of Questions	Marks
1	16	34	3	3	3	9	4	28	10	40
2	15	32	2	2	3	9	2	14	7	25
3	13	27	2	2	2	6	2	14	6	22
4	14	30	2	2	2	6	4	28	8	36
Total	58	123	9	9	10	30	12	84	31	123

Blue Print

Cognitive Level Mark Distribution

Cognitive Level	Marks	% of Marks
Remembering	8	7
Understanding	102	83
Applying	13	10
Analysing		
Evaluating		
Creating		
Total	123	100

SCHEME OF VALUATION

(Scoring Indicators)- Set I

Relational Database Management System

Part A

1. The collection of information stored in a database at a particular moment is called database instance.
2. The overall design of a database is called the database schema.
3. Set of allowed values for an attribute is referred to as domain. For example, a domain of month-of-year are January, February.... December, as possible values,
4. Each row of a relation is known as tuple
5. SQL is Structured Query Language, which is a computer language for storing, manipulating and retrieving data stored in a relational database.
6. CREATE TABLE table_name (. column1 datatype, column2 datatype, column3 datatype, ...);
7. A trigger is a stored program invoked automatically in response to an event such as insert, update, or delete that occurs in the associated table.
8. A cursor is a temporary work area created in the system memory when a SQL statement is executed. This temporary work area is used to store the data retrieved from the database, and manipulate this data.
9. A primary is a column or set of columns in a table that uniquely identifies tuples (rows) in that table.

Part B

1.Data Redundancy

Data Redundancy occurs when the same data are stored unnecessarily at different places. This issue is eliminated in DBMS by storing data in centralized location rather than being created by individual users and for each application.

2. Data Inconsistency

In traditional file system storage, the changes made by one user in one application doesn't update the changes in other application, provided both contains the same set of details. But in case of DBMS systems even though there is a single repository of data, accessed by many users, it will be updated continuously and hence the data remains consistent.

3. Data Sharing-

Data Sharing is the primary advantage of Database management systems. DBMS system allows users and applications to share Data with multiple applications and users.

4. Data Searching

Searching and retrieving of data is very easy in DBMS systems. Multiple information can be accessed from database servers with the help of small queries.

5. Data Security

DBMS systems provide a strong framework to protect data privacy and security. DBMS ensures that only authorized users have access to data and it has many methods to define access privileges.

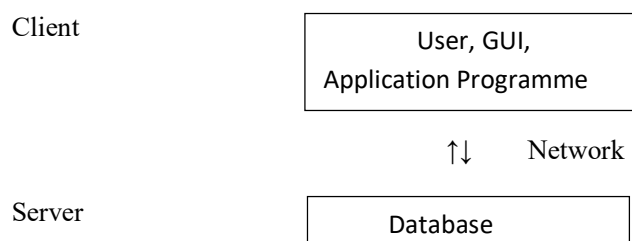
6. Data Concurrency

Data concurrency means that many users can access data at the same time. In DBMS, there will be locking mechanism (software) that prevents the same set of data from being changed by two people at the same time.

7. Data Integration-Data integration is a process of combining the data residing at different locations and present the user with a unified view of data.

8. Data Backup and Recovery- In DBMS, it is possible to create backup for existing data. Hence, whenever there happens any system failure or technical problems, we can easily access and use back up data.

2. In two-tier architecture, the **Database system is present at the server machine** and the user, GUI, and application programme is present at the client machine. These two machines are connected with each other through a reliable network as shown in the above diagram.



Whenever client machine makes a request to access the database present at server using a query language like SQL, the server process the request on the database and returns the result back to the client.

3.

Physical Data Independence	Logical Data Independence
Physical storage structure can be changed without affecting the logical schema	Logical schema can be changed without affecting the external schema
For improving performance	Structure of Database got changed
Not difficult to implement	Difficult to implement
Provide immunity to conceptual and external schema	Provide immunity to external schema and application programs

4. Key Constraint:- An attribute that can uniquely identify a tuple in a relation is called the key of the table. The value of the key attribute should be unique, ie no two tuple should have identical key attribute. Also, it should not hold null values.

Example: In the given table, CustomerID is a key attribute of Customer Table. It is most likely to have a single key for one customer, CustomerID =1 is only for the CustomerName =" Google".

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Domain Constraints: Every attribute is bound to have a specific range of values. For example, age cannot be less than zero and telephone numbers cannot contain a digit outside 0-9.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

5.

Dependency Preservation

During decomposition, we need to split a bigger relation to smaller relations. It is natural that the functional dependency which was present in the bigger relations also get split in to smaller relation. Each functional dependency $X \rightarrow Y$ should appear at least in one of the smaller relations or could be inferred from the dependency that occurs in smaller relations.

Decomposition

Decomposition is the process of splitting a relation in to smaller relations.

If R is a relation with A1,A2, A3, etc, as its attributes, then decomposition means splitting R in to smaller tables ie R1,R2.....Rn, where union of all these smaller tables results in R

6.

DESIGN GUIDELINES FOR RELATIONAL SCHEMA (Any three)

- 1.Semantics of the Attributes
- 2.Reducing the Redundant Value in Tuples.
- 3.Reducing Null values in Tuples.
- 4.Dissallowing spurious Tuples.

1. Semantics of the Attributes

Whenever we are going to form relational schema there should be some meaning among the attributes. This meaning is called semantics. This semantics relates one attribute to another with some relation.

Example :

SID	Student Name	Semester
-----	--------------	----------

2. Reducing the Redundant Value in Tuples

Data Redundancy may happen due to the following anomalies in tables

- Insertion anomalies
- Deletion anomalies
- Updation anomalies

Design in such a way that no insertion, deletion, modification anomalies will occur

3. Reducing Null values in Tuples.

Relations should be designed such that their tuples will have as few NULL values as possible

Attributes that are NULL frequently could be placed in separate relations (with the primary key)

4. Disallowing spurious Tuples

Bad designs for a relational database may result in erroneous results for certain JOIN operations

7.

SQL provides some basic inbuilt data types which are mentioned below

- **Numeric:** Numeric data types are used to represent decimal or floating-point numbers. It is generally declared in following manner
Numeric(i,j) where i represents the total number of digits before decimal and j represents the total number of digits after decimal. Example – Numeric (3,2) can hold value like 123.02 and 145.67. Values like 123.345 and 4518.34 can't be allowed
- **Integer:** Integer is used to represent data like numbers without decimals and is declared as int(10)
- **Character String:** Character string data types can be of fixed length or varying length. It is represented as CHAR(N), CHARACTER (N), VARCHAR(N), etc. when we use varchar it accepts variable characters.
- **Date:** Date datatype is used to represent dates. It is represented as YYYY-MM-DD and the components are year, month and day respectively.
- **Time Data Type:** Time data type has three positions and is used to specify time. Its components are hours, minutes and seconds in the following form HH:MM:SS
- **Time Stamp:** This data type is used to represent both date and time. It has 6 components, which are year, month, date, hour, minutes, and seconds in the following format – YYYY-MM-DD HH:MM:SS
- **Boolean Data Type:** This data type traditionally has two values which are true and false. In SQL Boolean data type has a third value which is **unknown**.
- **Bit String:** SQL also provides bit string data type which can be of fixed or varied length.

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

CREATE INDEX Example

The SQL statement below creates an index named "idx_lastname" on the "LastName" column in the "Persons" table:

```
CREATE INDEX idx_lastname ON Persons (LastName);
```

DROP INDEX

The DROP INDEX statement is used to delete an index in a table.

Syntax: DROP INDEX *index_name* ON *table_name*;

Example

```
DROP INDEX table_name.index_name;
```

9.

Concurrent Executions

Concurrent execution of database transactions in a multi-user system means that any number of users can use the same database at the same time. As the execution of the database is made in parallel, concurrency control is needed in order to avoid inconsistencies in the database.

Advantages are:

- Increased processor and disk utilization, leading to better transaction throughput. For example one transaction will be using the CPU while another is reading from or writing to the disk
- Reduced average response time for transactions: short transactions need not wait behind long ones.

Anomalies/Problems in Concurrent Execution

If transactions are executed concurrently, then the following problems/anomalies may happen. Common anomalies include

1. Last Update: Consider two transactions T1 and T2. If a value in the database is read by T2 transaction before T1 updates it, then it can lead to last update problem.
2. Dirty Data: Second problem occurs when system fails after an update and before changes are made. This updated value is read by another transaction and this is known as dirty data problem.
3. Unrepeatable Read: It is a problem which occur when a transaction has to read a data twice. In between these two reads another transaction update the data which lead to 2 values for same data. This is known as unrepeatable read.

10.

Committed – In this state, all the transactions are permanently saved to the database. This step is the last step of a transaction, if it executes without fail.

Aborted – If a transaction is failed to execute, then the database recovery system will make sure that the database is in its previous consistent state. If not, it brings the database to consistent state by aborting or rolling back the transaction. If the transaction fails in the middle of the transaction, all the executed transactions are rolled back to its consistent state before executing the transaction. Once the transaction is aborted it is either restarted to execute again or fully killed by the DBMS.

Part C

III.

Database Users

Database users are the one who really use and take the benefits of database. There will be different types of users depending on their need and way of accessing the database.

Native Users - these are the users who use the existing application to interact with the database. They are end users of database who are unaware of the presence of database system. For example, online library system, ticket booking systems, etc.

Application Programmers - They are the developers of application programs or user interfaces. They interact with the database by means of application programs/user interface written using high level programming languages like C, C++, JAVA, Python, etc.

Application programs communicate with the database with the help of queries. For example, writing a C program to generate the report of employees who are working in particular department will involve a query to fetch the data from database.

Sophisticated Users - They are database developers, who interact with the database system without writing application programs. They directly interact with the database by means of query language like SQL (Structured Query Language). These users will be scientists, engineers, analysts who thoroughly study SQL and DBMS to apply the concepts for their requirement.

Database Administrators- Database Administrator (DBA) is a person/team who defines the schema and also controls the 3 levels of database.

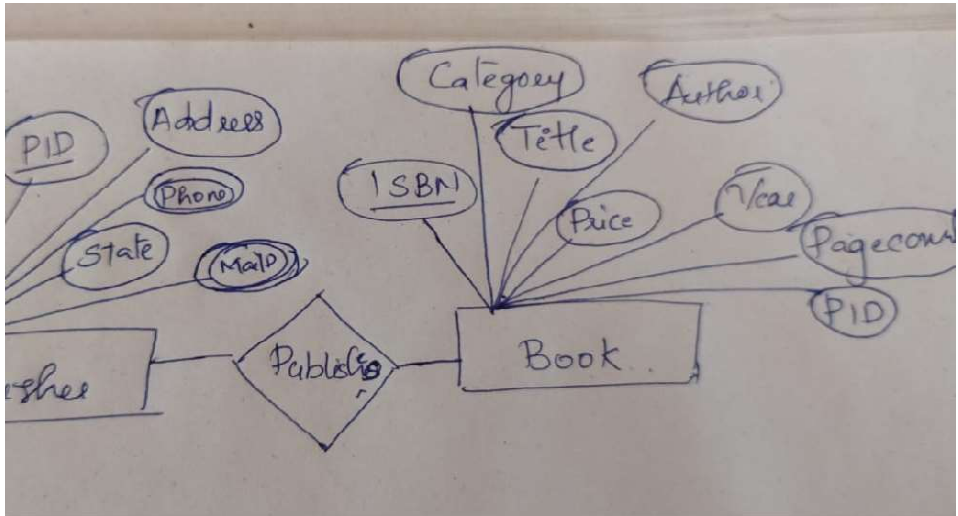
- Will create a new account id and password for the user if he/she need to access the data base.
- Responsible for providing security to the data base and he allows only the authorized users to access/modify the data base.
- Monitors the recovery and back up of data.

Query Processor

Query Processor is a component of DBMS which processes queries from users or programmers. To be clear, it translates statements in a query language into low-level instructions that can be processed by the system. The Query processor includes the following components.

- DDL Interpreter: It processes the DDL statements into a set of tables containing meta data (data about data).
- DML Compiler: DML Compiler translates DML statements into low-level instruction that is understandable by Query evaluation engine.
- Query Evaluation Engine - Query evaluation engine executes low level instructions generated by the DML compiler.

IV



V

Union

Pub_Id	Pub_Name
P01	Pearson Education
P02	Tata McGraw Hill
P03	Prentice Hall India
P04	Elsevier
P05	Penram International
P06	Bright Publication

Intersection

Pub_Id	Pub_Name

P01	Pearson Education
P02	Tata McGraw Hill

Difference

Pub_Id	Pub_Name
P05	Penram International
P06	Bright Publication

VI.

Select Operation

Select operation is used to select a subset of tuple from a relation that satisfies a particular selection criterion. In simple words, select operation is used to select certain rows from the table which satisfies the select condition. The symbol σ is used to denote select operation.

Syntax: σ <Selection Condition> R

R represents the name of the table.

Rename Operation

The rename operation allows us to rename the output relation. 'rename' operation is denoted with small Greek letter rho ρ .

Notation – ρ (T, σ <Selection Condition> R)

R represents the name of the table and T represents the new name of the table.

Cartesian product(X)

This type of operation is helpful to merge columns from two relations. It is also known as Cross Product or Cross Join and is used to combine information of 2 tables. If R & S are two relations then $R \times S$ will give another table T. Number of attributes in T will be equal to the sum of attributes of R & S and number of tuples in T will be equal to number of tuples in R multiplied by number of tuples in S.

VII. Describe the concept of nested queries in SQL with example.

Query written inside a query is called as Nested Query.

There are mainly **two types of nested queries**:

Independent Nested Queries: In independent nested queries, query execution starts from innermost query to outermost queries. The execution of inner query is independent of outer query, but the result of inner query is used in execution of outer query. Various operators like IN, NOT IN, ANY, ALL etc are used in writing independent nested queries.

Select ISBN,title,category from Book where PID=any(select PID from Publisher where Pname='ABC');

Book, Publisher: Relations

ISBN,title,category : Attributes of Book

Pname – Attribute of Publisher

Co-related Nested Queries: In co-related nested queries, the output of inner query depends on the row which is being currently executed in outer query.

Select * from Publisher where exists (Select * from Book where Publisher.PID=Book.PID);

VIII.

(i) Create table STUDENT (Regno int(10) as primary key, name varchar(30), branch varchar(30), semester varchar(10), mark int(10), grade varchar(10));

(ii)

Insert into STUDENT values(1111,'Asha','EC','S6',85,'S');

Insert into STUDENT values(1112,'Aathira','CB','S6',70,'A');

Insert into STUDENT values(1113,'Anoop','CT','S4',60,'B');

(iii)

Update STUDENT set grade='B' where Regno=1111;

(iv)

Select max(mark) from STUDENT group by branch;

Select min(mark) from STUDENT group by branch;

(v)

Select regno, name, branch, semester from STUDENT where mark=(select max(mark) from STUDENT);

IX. Explain stored procedures and functions

Stored Procedures and Functions

Stored Procedure is a prepared SQL code that can be saved, and the code can be used over and over again. If there is an SQL query that can write over and over again, save it as a stored procedure and then just call it and execute it. Stored Procedure are also known as Persistent Stored Module (PSM), as the data is persistently stored in the DBMS.

Stored Procedure Syntax CREATE PROCEDURE procedure_name As Sql_statement Go;	Executing a Stored Procedure EXEC procedure_name
---	--

Stored Function : A stored function is a set of SQL statements that perform some operation and return a single value.

The syntax for CREATE FUNCTION statement in Mysql is:

CREATE FUNCTION function_name(func_parameter1, func_parameter2, ..)

RETURN datatype [characteristics]

func_body

Parameters used:

- **function_name:** It is the name by which stored function is called.
- **func_parameter:** It is the argument whose value is used by the function inside its body.
- **datatype:** It is datatype of value returned by function.
- **func_body** is the set of Mysql statements that perform operation.

Basic Differences between Stored Procedure and Function in SQL Server

- The function must return a value but in Stored Procedure it is optional. Even a procedure can return zero or n values.
- Functions can have only input parameters for it whereas Procedures can have input or output parameters.
- Functions can be called from Procedure whereas Procedures cannot be called from a Function.

X. Explain the application of Embedded SQL statements

Embedded SQL statements and its application

Embedded SQL is the one which combines the high-level language with the DB language like SQL. It allows the application languages to communicate with DB and get requested result. It is a method of inserting inline SQL statements or queries into the code of a programming language. The high-level languages to which the SQL statements are inserted is known as host language. There are different host languages which support embedding SQL within it like C, C++, ADA, Pascal, FORTRAN, Java etc.

When SQL is embedded within a high-level language, the compiler processes the compilation in two steps. It first extracts all the SQL codes from the program and the pre-compiler will compile it for its syntax, correctness, execution path etc. Once pre-compilation is done, these executable codes are embedded into the higher-level code. Then the program compiler will compile the code and execute the code. Thus the compilation takes place in two steps – one for SQL and one for application language.

XI. Explain Serializability

Serializability

Schedule

When multiple transactions are running concurrently then there needs to be a sequence in the performance of operations because at a time only one operation can be performed on the database. This sequence of operations is known as **Schedule**.

Serial Schedule

In Serial schedule, a transaction is executed completely before starting the execution of another transaction. In serial schedule, a transaction does not start execution until the currently running transaction finished execution. This type of execution of transaction is also known as non-interleaved execution. The example we have seen above is the serial schedule. Example : two transactions T1 & T2 which are running concurrently, the transaction T2 does not start execution until the transaction T1 is finished.

Non-Serial Schedule -If interleaving of operations is allowed, then there will be non-serial schedule. In non-serial schedule, if the schedule is not proper, then the problems can arise like multiple update, uncommitted dependency and incorrect analysis.

Serializability - The main objective of serializability is to find non-serial schedules that allow transactions to execute concurrently without interference and produce a database state that could be produced by a serial execution.

XII. Explain the ACID properties of transaction.

Transaction Properties:

A transaction must have the following four properties, called ACID properties (also called ACIDITY of a transaction), to ensure that a database remains stable state after the transaction is executed:

1. **Atomicity.**
2. **Consistency.**
3. **Isolation.**
4. **Durability.**

Atomicity – This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none of the operations got executed. There must be no state in a database where a transaction is left partially completed.

Consistency – The database must remain in a consistent state after any transaction. No transaction should have any adverse effect on the data residing in the database. In other words, execution of a transaction must leave a database in either its prior stable state or a new stable state that reflects the new modifications (updates) made by the transaction.

If the transaction fails, the database must be returned to the state it was in prior to the execution of the failed transaction.

If the transaction commits, the database must reflect the new changes. Thus, all resources are always in a consistent state.

Isolation

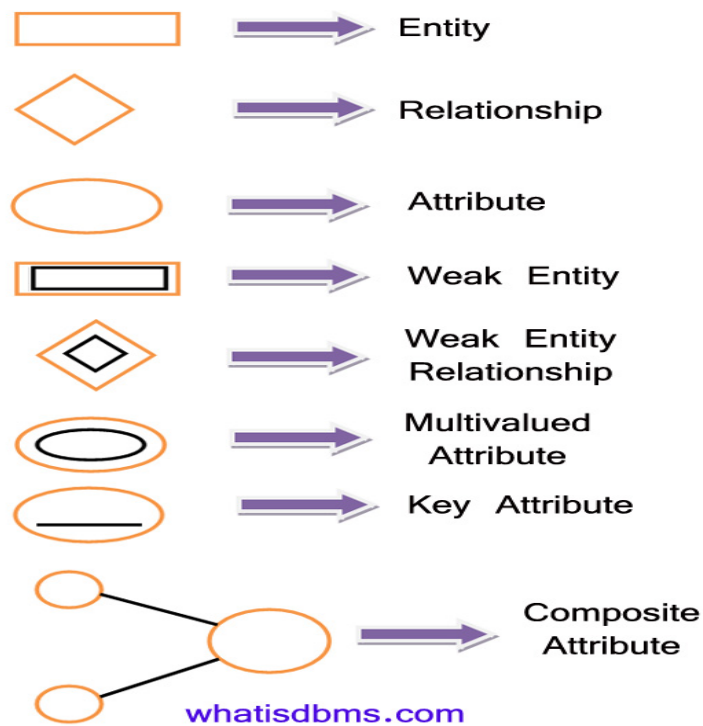
This property ensures that multiple transactions can occur concurrently without leading to the inconsistency of database state. Isolation property of a transaction means that the data used during the execution of a transaction cannot be used by a second transaction until the first one is completed. This property isolates transactions from one another. In other words, if a transaction T1 is being executed and is using the data item X, that data item cannot be accessed by any other transaction (T2.....Tn) until T1 ends.

Durability

The database should be durable enough to hold all its latest updates even if the system fails or restarts. The changes made by a transaction cannot be lost by either a system failure or by the erroneous operation of a faulty transaction. When a transaction is completed, the database reaches a consistent state and that state cannot be lost, even in the event of system's failure. If a transaction fails before the data could be written on to the disk, then that data will be updated once the system springs back into action.

XIII. Specify symbols and the meaning of ER Model Notations.

E R Diagram Symbols



Types of Attributes

Attributes can be classified into various types which are explained below.

Simple/Key Attribute: - The attribute which is used to uniquely identify an entity is known as a key attribute. For example, student roll number can uniquely identify a student from a set of students.

Composite Attribute: - Attribute which can be divided further is known as a composite attribute. For example, in student entity, the student address is a composite attribute which can be further divided into pin code, state, country.

3. Multivalued attribute: An attribute that can hold multiple values is known as a multivalued attribute. For example – A person can have more than one phone numbers so the phone number attribute is multivalued.

4. Derived attribute:

A derived attribute is one whose value is dynamic and derived from another attribute. For example – Person age is a derived attribute as it changes over time and can be derived from another attribute (Date of birth).

XIV

Relationship:- Association between two or more entities is considered as a relationship.

Relationship Types

The association between two or more entity is known as relationship. There are four types of relationships:

1. One to One
2. One to Many
3. Many to One
4. Many to Many

1. One to One Relationship

When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. For example, a person can hold only one passport and a passport can be issued to a single person.

2. One to Many Relationship

When a single instance of an entity is associated with more than one instances of another entity then it is called one-to-many relationship. For example – a customer can place many orders but a single order cannot be placed by many customers.

3. Many to One Relationship

When more than one instances of an entity is associated with a single instance of another entity then it is called many to one relationship. For example – many students can study in a single college but a student cannot study in many colleges at the same time.

4. Many to Many Relationship

When more than one instances of an entity is associated with more than one instances of another entity then it is called many-to-many relationship. For example, a student can be assigned with many projects and a project can be assigned to many students.

MODEL QUESTION PAPER

RELATIONAL DATABASE MANAGEMENT SYSTEM - SET II

(Maximum marks: 75)

(Time: 3 hours)

PART A

I. Answer all questions in one word or one sentence. Each question carries one mark.

(9 x 1 = 9 marks)

1	Define field in database	M 1.01	R
2	Define data independence	M 1.01	R
3	Define Functional dependency	M 2.04	R
4	Quote any two unary relational algebra operations	M 2.02	A
5	State any one feature of SQL	M 3.01	U
6	State the use of numeric data type	M 3.01	U
7	Recall the term 'Transaction'	M 4.03	R
8	State any two properties of transaction	M 4.03	U
9	Quote the full form of 'ER' in ER model	M 1.03	U

PART B

II Answer any eight questions from the following. Each question carries 3 marks

(8 x 3 = 24marks)

1	Demonstrate weak entity set with an example.	M 1.03	U
---	--	--------	---

2	Explain three tier client/server architecture	M 1.02	U
3	Elaborate the terms i) Specialisation ii) Generalisation	M 1.03	U
4	Define normalization. Explain 1NF with 'example'	M 2.04	A
5	Explain the following operations with examples: (i) Union (ii) Difference	M 2.02	U
6	Explain (i) Null Constraint (ii) Default Constraint	M 2.01	U
7	Explain any three aggregate functions	M 3.02	A
8	Explain the commands (i) ALTER (ii) DROP TABLE	M 3.02	A
9	Elaborate on the following transaction states (i) Failed (ii) Terminated	M 4.03	U
10	Explain on Stored Procedures and Functions	M 4.01	U

PART C

Answer all questions. Each question carries seven marks

(6x7=42 marks)

III	List and briefly explain any seven advantages of DBMS. OR	M1.01	U
IV	With a neat sketch explain three schema architecture of DBMS	M1.01	U
V	Describe (i) Disk Storage (ii) Storage Manager OR	M1.02	U
VI	Explain briefly the concept of Database Languages DDL, DML, DCL	M1.02	U
VII	Describe briefly about any three binary operators in relational algebra OR	M2.02	U
VIII	Describe how the mapping from ER model to relational model is achieved outlining the steps involved in the process.	M2.01	U
IX	Write the general syntax of the command for creating a table for the following relation - Employee (ENO, ENAME, DEPARTMENT, BASICPAY, DA, HRA, GROSSPAY DEDUCTION, NETPAY). Assign suitable data types and set the Primary Key. Write Queries for the following: (i) Calculate DA as 50% of Basic Pay, Gross pay as Basic pay + DA + HRA and Net pay as Gross pay -Deductions (iv) Find the number of employees working in 'Computer' Department. (v) Retrieve the maximum, minimum and average gross salary of each department. OR	M3.02	A
X	Define Query Optimization. Describe the three types of query	M3.02	A

	optimization methods.		
XI	Explain: (i) Features of Embedded SQL (ii) Concurrent Execution of transaction.	M4.02, 4.03	U
	OR		
XII	Explain the stages of Transaction	M4.03	U
XIII	Elaborate the ACID Properties of transaction.	M4.03	U
	OR		
XIV	Elaborate the concept of Trigger.	M4.01	U

Question Wise Analysis

Course : Relational Database Management System

Q. No	Module Outcome	Cognitive Level	Marks	Time
I.1	M 1.01	R	1	1 min
I.2	M 1.01	R	1	1 min
I.3	M 2.04	R	1	1 min
I.4	M 2.02	A	1	1 min
I.5	M 3.01	U	1	1 min
I.6	M 3.01	U	1	1 min
I.7	M 4.03	R	1	1 min
I.8	M 4.03	U	1	1 min
I.9	M 1.03	U	1	1 min
II.1	M 1.03	U	3	6 min
II.2	M 1.02	U	3	6 min
II.3	M 1.03	U	3	6 min
II.4	M 2.04	A	3	6 min
II.5	M 2.02	U	3	6 min

II.6	M 2.01	U	3	6 min
II.7	M 3.02	A	3	6 min
II.8	M 3.02	A	3	6 min
II.9	M 4.03	U	3	6 min
II.10	M 4.01	U	3	6 min
III	M 1.01	U	7	20 min
IV	M 1.01	U	7	20 min
V	M 1.02	U	7	20 min
VI	M 1.02	U	7	20 min
VII	M 2.02	U	7	20 min
VIII	M 2.01	U	7	20 min
IX	M 3.02	A	7	20 min
X	M 3.02	A	7	20 min
XI	M 4.02 & M 4.03	U	7	20 min
XII	M 4.03	U	7	20 min
XIII	M 4.03	U	7	20 min
XIV	M 4.01	U	7	20 min

Course : Relational Database Management System

Blue Print

Mark Distribution

Module	hr / Module	$(\sum H_i)$	Type of Questions			
			Part A	Part B	Part C	Total

			No of Questions	Marks	No of Questions	Marks	No of Questions	Marks	No of Questions	Marks
1	16	34	3	3	3	9	4	28	10	40
2	15	32	2	2	3	9	2	14	7	25
3	13	27	2	2	2	6	2	14	6	22
4	14	30	2	2	2	6	4	28	8	36
Total	58	123	9	9	10	30	12	84	31	123

Blue Print

Cognitive Level Mark
Distribution

Cognitive Level	Marks	% of Marks
Remembering	4	3
Understanding	113	92
Applying	6	5
Analysing		
Evaluating		
Creating		
Total	123	100

SCHEME OF VALUATION
(Scoring Indicators)- Set II

Relational Database Management System

Part A

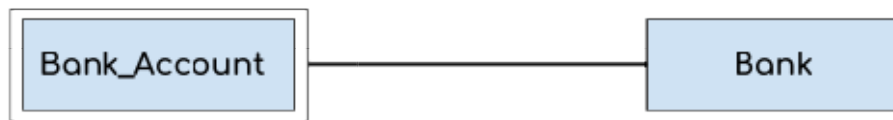
1. Fields are defined as the columns of the database table which stores similar kind of data
2. The ability to modify schema definition in one level without affecting the schema definition at higher levels is known as data independence.
3. A functional dependency (FD) is a relationship between two attributes, typically between the Primary Key and other non-key attributes within a table.
4. Select, Project, Rename, Assign (Any two)
5. SQL is used to access data from relational database management systems or SQL can execute queries against the database.
6. Numeric data types are used to represent decimal or floating-point numbers

7. A transaction can be defined as a logical unit of work on the database. A transaction generally represents any change in a database.
8. Atomicity, Consistency, Isolation, Durability (Any two)
9. Entity-Relationship model

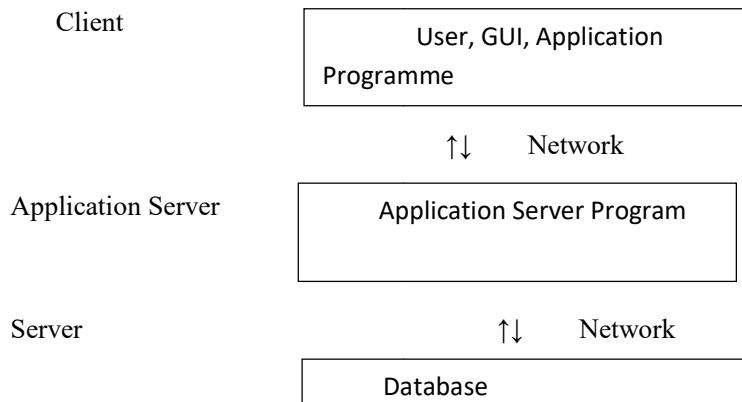
Part B

1. An entity that does not have a key attribute of its own is called weak entity.

The weak entity is represented by a double rectangle. For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



2. In three-tier architecture also, the Database system is present at the server machine and the user, GUI, and application programme is present at the client machine. The difference in this case is that another layer is present between the client machine and server machine and is known as application server. The client communicates with the application server which in turn communicates with the database server and gives the result.



3.

1. Generalization

- Generalization is the process of combining the sub-entities to a main entity, which contains all the properties of the sub entities.
- It is a bottom up approach, in which lower level entities combine to form a higher-level entity.
- Generalization is the reverse process of Specialization.

- For example: In the above example, Tiger, Lion, Elephant can all be generalized as Animals.

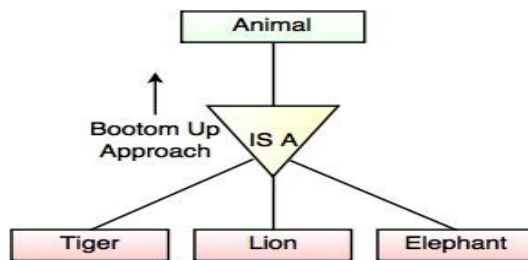


Fig. Generalization

-

2. Specialization

- Specialization is the process of dividing a main entity into sub entities based on their characteristic.
- It is a top down approach, in which one higher entity can be broken down into two lower level entity.
- It defines one or more sub class for the super class and forms a superclass/subclass relationship.

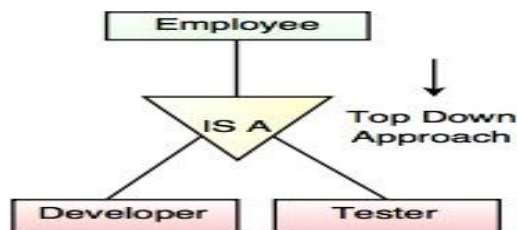


Fig. Specialization

-

For example: In the above example, Employee can be specialized as Developer or Tester, based on what role they play in an Organization.

4.

1st Normal Form

A database is in first normal form if it satisfies the following conditions:

- Contains only atomic values
- There are no repeating groups

An atomic value is a value that cannot be divided. For example, in the table shown below, the values in the [Color] column in the first row can be divided into "red" and "green", hence [TABLE_PRODUCT] is not in 1NF.

A repeating group means that a table contains two or more columns that are closely related. For example, a table that records data on a book and its author(s) with the following columns: [Book ID], [Author 1], [Author 2], [Author 3] is not in 1NF because [Author 1], [Author 2], and [Author 3] are all repeating the same attribute.

1st Normal Form Example

Unnormalized Table Example

TABLE_PRODUCT

Product ID	Color	Price
1	red, green	15.99
2	yellow	23.99
3	green	17.50
4	yellow, blue	9.99
5	red	29.99

This table is not in first normal form because the [Color] column can contain multiple values. For example, the first row includes values "red" and "green." To bring this table to first normal form, we split the table into two tables and now have the resulting tables:

Now first normal form is satisfied, as the columns on

each table all hold just one value.

TABLE_PRODUCT_PRICE

Product ID	Price
1	15.99
2	23.99
3	17.50
4	9.99
5	29.99

TABLE_PRODUCT_COLOR

Product ID	Color
1	red
1	green
2	yellow
3	green
4	yellow
4	blue
5	red

5.

Difference

Difference is denoted by – symbol. It is used to find out tuples which are present in one relation but not in other relation. If there are two relations R1 and R2 and to select all those tuples(rows) that are present in Relation R1 but not present in Relation R2, this can be done using Set difference $R1 - R2$.

Consider two tables A and B

Table A

F-Name	L-Name
Reghu	T
Riju	M
Mohan	S
Venu	R
Mary	D

Table B

F-Name	L-Name
Sanjay	S
John	M
Anitha	S
Reghu	T
Venu	R

A-B

F-Name	L-Name
Riju	M
Mohan	S
Mary	D

Union Operation

UNION is symbolized by $\cup$ symbol. It includes all tuples that are in tables A or in B. It also eliminates duplicate tuples. So, set A UNION set B would be expressed as: The result $\leftarrow A \cup B$

For a union operation to be valid, the following conditions must hold -

- Both relations should have the same number of attributes.
- Attribute domains need to be compatible.
- Duplicate tuples should be automatically removed.

Example: Consider the following tables.

Consider the following tables.

Table A		Table B	
column 1	column 2	column 1	column 2
1	1	1	1
1	2	1	3

$A \cup B$ gives

Table $A \cup B$	
column 1	column 2
1	1
1	2
1	3

6.

1. NULL Constraint – Ensures that a column cannot have NULL value.

NULL constraint restricts a column from having a NULL value. Once **NOT NULL** constraint is applied to a column, you cannot pass a null value to that column. It enforces a column to contain a proper value.

Example using NOT NULL constraint

```
CREATE TABLE Student (s_id int NOT NULL, Name varchar (60), Age int);
```

The above query will declare that the **s_id** field of **Student** table will not take NULL value.

2. DEFAULT Constraint – Provides a default value for a column when none is specified.

Default value for an attribute can be specified by writing the default clause for an attribute in create table command. When default value is mentioned entire column will take same value.

Syntax - Attribute name, datatype, Default 'value'

7.

COUNT Function: The COUNT function returns the total number of values in the specified field.

MIN function: The MIN function returns the smallest value in the specified table field.

MAX function: It returns the largest value from the specified table field.

SUM function returns the sum of all the values in the specified column.

AVG function: AVG function returns the average of the values in a specified column.

(Simple examples can be given)

8

ALTER command is used for altering the table structure, such as,

- to add a column to existing table
- to rename any existing column
- to change datatype of any column or to modify its size.
- to drop a column from the table.

Syntax : ALTER TABLE table_name ADD(column_name datatype);

The **DROP TABLE** statement is used to drop an existing table in a database.

Syntax. DROP TABLE table_name

9

Failed – A transaction is said to be in a failed state if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.

Terminated: When the transaction leaves the system, it enters the terminated state. Termination state can be attained either from failure or committed state.

10

Stored Procedures and Functions

Stored Procedure is a prepared SQL code that can be saved, and the code can be used over and over again. If there is an SQL query that can write over and over again, save it as a stored procedure and then just call it and execute it. Stored Procedure are also known as Persistent Stored Module (PSM), as the data is persistently stored in the DBMS.

Stored Procedure Syntax CREATE PROCEDURE procedure_name As Sql_statement Go;	Executing a Stored Procedure EXEC procedure_name
---	--

Stored Function : A stored function is a set of SQL statements that perform some operation and return a single value.

The syntax for CREATE FUNCTION statement in Mysql is:

```
CREATE FUNCTION function_name(func_parameter1, func_parameter2, ..)
```

```
    RETURN datatype [characteristics]
```

```
func_body
```

Parameters used:

- function_name: It is the name by which stored function is called.
- func_parameter: It is the argument whose value is used by the function inside its body.
- datatype: It is datatype of value returned by function.
- func_body is the set of Mysql statements that perform operation.

Basic Differences between Stored Procedure and Function in SQL Server

- The function must return a value but in Stored Procedure it is optional. Even a procedure can return zero or n values.
- Functions can have only input parameters for it whereas Procedures can have input or output parameters.
- Functions can be called from Procedure whereas Procedures cannot be called from a Function.

Part C

III.

Advantages of Database Management System

1. Data Redundancy

Data Redundancy occurs when the same data are stored unnecessarily at different places. This issue is eliminated in DBMS by storing data in centralized location rather than being created by individual users and for each application.

2. Data Inconsistency

In traditional file system storage, the changes made by one user in one application doesn't update the changes in other application, provided both contains the same set of details. But in case of DBMS systems even though there is a single repository of data, accessed by many users, it will be updated continuously and hence the data remains consistent.

3. Data Sharing-

Data Sharing is the primary advantage of Database management systems. DBMS system allows users and applications to share Data with multiple applications and users.

4. Data Searching

Searching and retrieving of data is very easy in DBMS systems. Multiple information can be accessed from database servers with the help of small queries.

5. Data Security

DBMS systems provide a strong framework to protect data privacy and security. DBMS ensures that only authorized users have access to data and it has many methods to define access privileges.

6. Data Concurrency

Data concurrency means that many users can access data at the same time. In DBMS, there will be locking mechanism (software) that prevents the same set of data from being changed by two people at the same time.

7. Data Integration-Data integration is a process of combining the data residing at different locations and present the user with a unified view of data.

8. Data Backup and Recovery- In DBMS, it is possible to create backup for existing data. Hence, whenever there happens any system failure or technical problems, we can easily access and use back up data.

IV.

Three Schema Architecture

Following are the three levels of database architecture,

1. Physical Level
2. Conceptual/Logical Level
3. External Level

1. Physical Level

- Physical level describes the physical storage structure of data in database.
- It is also known as Internal Level.
- This level represents the physical storage of data.
- At lowest level, it is stored in the form of bits (with the physical addresses on the secondary storage device).
- At highest level, it can be viewed in the form of files.

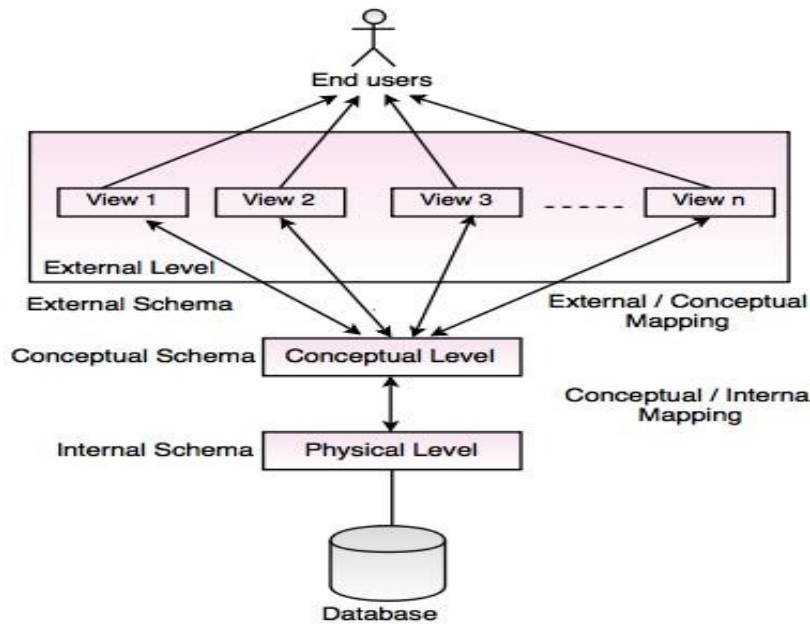


Fig. Three Level Architecture of DBMS

2. Logical/Conceptual Level

- Conceptual level describes the entire content of the database and the relationship between these data.
- This level is more structured and generally the database administrator uses this level

3. External Level

- External level is related to the data which is viewed by individual end users.
- External view describes the segment of the database that is required for a particular user group and hides the rest of the database from that user group.

V.

Storage Manager

The storage manager is a program module that manages the data in the database. Program module acts as an interface between low level data in the database and application programs/queries submitted to the system. The Storage manager is responsible for efficient data storage and fast data retrieval.

The storage manager components include the following.

- **Authorisation/Integrity Manager:** - Checks the authority of the user to access the data
- **File Manager:** - Manages the data stored in the disk, by providing adequate space on disk storage and manages the data structures used to represent information
- **Transaction Manager:** - Transaction manager ensures that the database remains in a consistent state and allowing concurrent transactions to proceed without conflicting. If there are multiple users accessing the database at the same time, it makes sure that there is no data loss or data mismatch happens between these transactions.

- **Buffer Manager:** - It is responsible for fetching the data from disk storage and saving it in the main memory. It decides on what data or which data to be cached.

Disk Storage

Disk storage is the place where the data is stored. It's further divided into 4 components. They are

- 1) Data
As database is a collection of interrelated data, one of the major components is the data itself.
- 2) Data Dictionary/ Metadata
Data dictionary also known as metadata stores information about data.
- 3) Data Indices (Index)
It contains information about the location of the data and is used to access data location quickly.
- 4) Statistical Data
Statistical data contains statistical information about the data in a database. This information will be used by the Storage Manager component of the database to decide which data is to be cached.

VI. Data Definition Language (DDL) –Data Definition Language (DDL) are commands used to create structure of the database. It is a query used to createfiles, databases, data dictionary and tables within databases.

Query- statement requesting creation of database schema or retrieval of information

The following table gives an overview about usage of DDL statements in SQL

S.No	Need and Usage	The SQL DDL statement
1	Create schema objects	CREATE
2	Alter schema objects	ALTER
3	Delete schema objects	DROP
4	Reneme schema objects	RENAME

Data Manipulation Language (DML)

It is a language that provides a set of operations to support the basic data manipulation operations on the data held in the databases. It allows users to insert, update, delete and retrieve data from the database.

The following table gives an overview about the usage of DML statements in SQL:

S. No	Need and Usage	The SQL DML statement
1	Remove rows from tables or views	DELETE
2	Add new rows of data into table or view	INSERT
3	Retrieve data from one or more tables	SELECT
4	change column values in existing rows of a table or view	UPDATE

Data Control Language (DCL) –

DCL statements control access to data and the database using statements such as **GRANT** and **REVOKE**. A privilege can either be granted to a User with the help of GRANT statement. The

privileges assigned can be SELECT, ALTER, DELETE, EXECUTE, INSERT, INDEX etc. In addition to granting of privileges, it is also possible to revoke (taken back) the granted permission using REVOKE command.

The following table gives an overview about the usage of DCL statements in SQL:

S. No.	Need And Usage	Age
1	Grant and take away privileges and roles	Grant Revoke
2	Add a comment to the data dictionary	Comment

In practice, the data definition and data manipulation languages are not two separate languages. Instead they simply form parts of a single database language such as Structured Query Language (SQL).

VII.

Union Operation

UNION is symbolized by $\cup$ symbol. It includes all tuples that are in tables A or in B. It also eliminates duplicate tuples. So, set A UNION set B would be expressed as: The result $\leftarrow A \cup B$

For a union operation to be valid, the following conditions must hold -

- Both relations should have the same number of attributes.
- Attribute domains need to be compatible.
- Duplicate tuples should be automatically removed.

Example: Consider the following tables.

Consider the following tables.

Table A		Table B	
column 1	column 2	column 1	column 2
1	1	1	1
1	2	1	3

$A \cup B$ gives

Table $A \cup B$	
column 1	column 2
1	1
1	2
1	3

Intersection

An intersection is defined by the symbol $\cap$

$A \cap B$: Defines a relation consisting of a set of all tuple that are in both A and B.

Table $A \cap B$	
column 1	column 2
1	1

Difference

Difference is denoted by $-$ symbol. It is used to find out tuples which are present in one relation but not in other relation. If there are two relations R_1 and R_2 and to select all those tuples (rows) that are present in Relation R_1 but not present in Relation R_2 , this can be done using Set difference $R_1 - R_2$.

Consider two tables A and B

Table A

F-Name	L-Name
Reghu	T
Riju	M
Mohan	S
Venu	R
Mary	D

Table B

F-Name	L-Name
Sanjay	S
John	M
Anitha	S
Reghu	T
Venu	R

$A - B$

F-Name	L-Name
Riju	M
Mohan	S
Mary	D

VIII

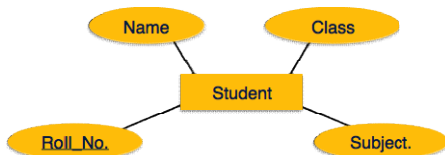
Mapping of ER Model to Relational Model

Both ER model and relational model are real time representation of database and hence ER model can be mapped to a relational model. Following steps are used to map the ER model to relational model.

1. Mapping of Regular Entity

For mapping the regular entity to relational model, the following steps are to be followed.

- Name of the entity in the ER model becomes the name of the relation
- Attributes of entity become the attribute of the relation
- Each instance of the entity becomes the tuple of the relation
- Primary key of the ER diagram becomes the primary key of the relation
- Insert the data in to the relation or the table created



2. Mapping of Entity with Composite Attribute

To represent composite attributes of ER model to relational model, the following steps are followed

- Follow all steps for mapping a regular entity from ER model to relational model
- Column for composite attribute is subdivided in to separate columns, and then include each sub attribute in the relation.

3. Mapping of Entity with Multivalued Attribute

To represent multivalued attributes of ER model to relational model, the following steps are followed

- Follow all steps for mapping a regular entity from ER model to relational model
- Additional number of rows are inserted in the table for representing multiple values of an attribute. Number of rows to be inserted will be equal to the number of values for an attribute.

4. Mapping of Weak Entity

To represent weak entity of ER model to relational model, the following steps are followed

- Follow all steps for mapping a regular entity from ER model to relational model
- The attribute which is the primary key of the strong entity is added as attribute of the weak entity in relational model
- Primary key of the table or relation is combination of primary key of strong entity and the partial key of weak entity

5. Mapping of Binary Relationship

For mapping binary relationship of ER model to relational model the following steps are made.

- Create two relations or two tables by using the steps included in mapping ER model to relation model.

- To show binary relationship primary key on one table becomes an attribute of another table.

IX

Create table Employee (ENO int (10) primary key, ENAME varchar (30), DEPARTMENT varchar(50), BASICPAY int(20), HRA int(20));
 Insert into Employee values(101,'Sanjay','Electronics',20000,1250);
 Insert into Employee values(102,'Rahul','Computer',10000,1000);
 Insert into Employee values(103,'Mohan','Instrumentation',15000,1500);
 Alter table Employee add DA int(20), HRA int(10), GROSSPAY int(20), DEDUCTION int(20), NETPAY int(20);

Update Employee set DA=BASICPAY*50/100;
 Update Employee set GROSSPAY=BASICPAY+DA+HRA;
 Update Employee set NETPAY=GROSSPAY-DEDUCTION;

Select count(ENO) from Employee where DEPARTMENT='Computer';

Select max(GROSSPAY) from Employee group by DEPARTMENT
 Select min(GROSSPAY) from Employee group by DEPARTMENT
 Select avg(GROSSPAY) from Employee group by DEPARTMENT

X.

Query Optimization

Query optimization is a feature of many relational database management systems, which determines the efficient way to execute a query with different possible query plans.

Types of Query Optimization

Cost based Query Optimization: This is the simple and most commonly used method for query optimization. In this plan various possible execution plans for a query are generated and the least cost plan is selected for execution. Cost for executing query include-

- Cost for searching secondary storage
- Cost of storing intermediate result
- Computing cost
- Memory Usage cost
- Communication cost

Hence in this method, the execution plan which takes least cost is selected.

Heuristic Query Optimization: Another method for query optimization is to use heuristic approach. The idea here is to execute a query in a way as to reduce the intermediate storage requirements (Example: By using select and project operation before Join. This reduces the number of tuples available for join)

Semantic Query Optimization: This technique of query optimization is used along with other techniques mentioned above. Here database uses constraints to modify a query and execute it efficiently.

XI

Embedded SQL statements and its application

Embedded SQL is the one which combines the high-level language with the DB language like SQL. It allows the application languages to communicate with DB and get requested result. It is a method of inserting inline SQL statements or queries into the code of a programming language. The high-level languages to which the SQL statements are inserted is known as host language. There are different host languages which support embedding SQL within it like C, C++, ADA, Pascal, FORTRAN, Java etc.

When SQL is embedded within a high-level language, the compiler processes the compilation in two steps. It first extracts all the SQL codes from the program and the pre-compiler will compile it for its syntax, correctness, execution path etc. Once pre-compilation is done, these executable codes are embedded into the higher-level code. Then the program compiler will compile the code and execute the code. Thus the compilation takes place in two steps – one for SQL and one for application language.

Concurrent Executions

Concurrent execution of database transactions in a multi-user system means that any number of users can use the same database at the same time. As the execution of the database is made in parallel, concurrency control is needed in order to avoid inconsistencies in the database.

Advantages are:

- Increased processor and disk utilization, leading to better transaction throughput. For example one transaction will be using the CPU while another is reading from or writing to the disk
- Reduced average response time for transactions: short transactions need not wait behind long ones.

Anomalies/Problems in Concurrent Execution

If transactions are executed concurrently, then the following problems/anomalies may happen. Common anomalies include

4. Last Update: Consider two transactions T1 and T2. If a value in the database is read by T2 transaction before T1 updates it, then it can lead to last update problem.
5. Dirty Data: Second problem occurs when system fails after an update and before changes are made. This updated value is read by another transaction and this is known as dirty data problem.
6. Unrepeatable Read: It is a problem which occur when a transaction has to read a data twice. In between these two reads another transaction update the data which lead to 2 values for same data. This is known as unrepeatable read.

XII

Each transaction has following 5 states:

Active – This is the first state of transaction and here the transaction is being executed. READ and WRITE operations are issued at this stage.

Partially Committed – Partially Committed – When a transaction executes its final operation, it is said to be in a partially committed state. But data is still not saved to the database.

Committed – In this state, all the transactions are permanently saved to the database. This step is the last step of a transaction, if it executes without fail.

Failed – A transaction is said to be in a failed state if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.

Aborted – If a transaction is failed to execute, then the database recovery system will make sure that the database is in its previous consistent state. If not, it brings the database to consistent state by aborting or rolling back the transaction. If the transaction fails in the middle of the transaction, all the executed transactions are rolled back to its consistent state before executing the transaction. Once the transaction is aborted it is either restarted to execute again or fully killed by the DBMS.

Terminated: When the transaction leaves the system, it enters the terminated state. Termination state can be attained either from failure or committed state.

XIII

A transaction must have the following four properties, called ACID properties (also called ACIDITY of a transaction), to ensure that a database remains stable state after the transaction is executed:

5. Atomicity.
6. Consistency.
7. Isolation.
8. Durability.

Atomicity – This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none of the operations got executed. There must be no state in a database where a transaction is left partially completed.

Consistency – The database must remain in a consistent state after any transaction. No transaction should have any adverse effect on the data residing in the database. In other words, execution of a transaction must leave a database in either its prior stable state or a new stable state that reflects the new modifications (updates) made by the transaction.

If the transaction fails, the database must be returned to the state it was in prior to the execution of the failed transaction.

If the transaction commits, the database must reflect the new changes. Thus, all resources are always in a consistent state.

Isolation

This property ensures that multiple transactions can occur concurrently without leading to the inconsistency of database state. Isolation property of a transaction means that the data used during the execution of a transaction cannot be used by a second transaction until the first one is completed. This property isolates transactions from one another. In other words, if a transaction T1 is being executed and is using the data item X, that data item cannot be accessed by any other transaction (T2.....Tn) until T1 ends.

Durability

The database should be durable enough to hold all its latest updates even if the system fails or restarts. The changes made by a transaction cannot be lost by either a system failure or by the erroneous operation of a faulty transaction. When a transaction is completed, the database reaches a consistent state and that state cannot be lost, even in the event of system's failure. If a transaction fails before the data could be written on to the disk, then that data will be updated once the system springs back into action.

XIV

A trigger is a stored program invoked automatically in response to an event such as insert, update, or delete that occurs in the associated table. For example, you can define a trigger that is invoked automatically before a new row is inserted into a table.

The SQL standard defines two types of triggers: row-level triggers and statement-level triggers.

A row-level trigger is activated for each row that is inserted, updated, or deleted. For example, if a table has 100 rows inserted, updated, or deleted, the trigger is automatically invoked 100 times for the 100 rows affected.

A statement-level trigger is executed once for each transaction regardless of how many rows are inserted, updated, or deleted.

Advantages of triggers

- Triggers provide another way to check the integrity of data.
- Triggers handle errors from the database layer.
- As the triggers are invoked automatically before or after a change is made to the data in a table, there is no need to wait for the scheduled events to run
- Triggers can be useful for auditing the data changes in tables.

The **basic syntax of the CREATE TRIGGER statement:**

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE| DELETE}
ON table_name FOR EACH ROW
trigger_body;
```